National Institute of Technology Karnataka Surathkal
Department of Information Technology

# IT 252 DATABASE SYSTEMS

## Transaction Processing

Dr. Jayashree T R

# Course Outline

**Course Plan: Theory:**

**Part A: Parallel Computer Architectures**

Week 1: Introduction to to the course, highlighting the data and databases, and related basic concepts. Advantages and need for a database system.

Week 2: Attributes, tuples, relational schema, conceptual model, introduction to SQL-DML, DDL, creating relations/ tables, access and manipulate data.

Week 3,4: Data model, relational model, and SQL, relational data model and relational database constraints, SQL data definition and data types, specifying constraints, basic retrieval queries, complex quires, triggers, views, schema modification.

Week 5-6: ER model development, entity types/sets, attribute, relationship types/sets, simple employee database conceptual design using ER concepts, ER to relational mapping algorithm, EER model.

# Course Outline

Week 7,8 : Functional dependency definition, need of normalization, anomalies and redundant information in tuples, normalization steps (INF, 2NF, 3NF), BCNF, introduction to the higher normal forms.

Week 9: Procedural query languages: relational algebra and SQL-SELECT, PROJECT, RENAME, binary and JOIN operations with examples.
Non-Procedural query languages: relational calculus, and SQL-tuple relational calculus, existential a universal quantifier with examples.

Week 10,11 : Transaction management, schedule and serializability, concurrency control, 2-phase lock, recovery mechanism: undo/redo values.

Week 12,13: Disk storage- basic file structures, ordered/unordered. binary search, hashing, indexing, importance of indexes, primary and secondary indexing methods, clustered, B tree, B++ trees.

Week 14: Current trends in database system, introduction to data warehousing, data mining.

Practical: MySQL commands

Project: Team of 2 or 3 members

# Introduction to Transaction Processing Concepts and Theory

## Introduction to Transaction Processing

- Single-user DBMS (system): At most one user at a time can use the system
  - Example: home computer
- Multiuser DBMS (system): Many users can access the system (database) concurrently
  - Example: airline reservations system
- Concurrency:
  - Multiprogramming:
    - Allows operating system to execute multiple processes concurrently
    - Executes commands from one process, then suspends that process and executes commands from another process, etc.

Note: **Multiprogramming** is a concept related to **concurrent program execution**, but they are not exactly the same.

In Multiprogramming the CPU switches between programs (processes) quickly, giving the illusion that they are running simultaneously.

Concurrency is the ability of a system to execute multiple tasks at the same time (or make it appear as if they're running simultaneously). A single-core CPU can run concurrent tasks by switching between them quickly (e.g.: multitasking on your mobile phone).

**Parallelism:** is about executing **multiple tasks simultaneously**. It requires **multiple processing units** (like **multi-core processors**) where tasks are truly running at the same time. E.g.: Weather Forecasting, data processing, video rendering, ML based applications.

# Introduction to Transaction Processing

Concurrency:

- Interleaved processing: Concurrent execution of processes is interleaved in a single CPU.
- Parallel processing: Processes are concurrently executed in multiple CPUs.
  - Processes C and D in figure below

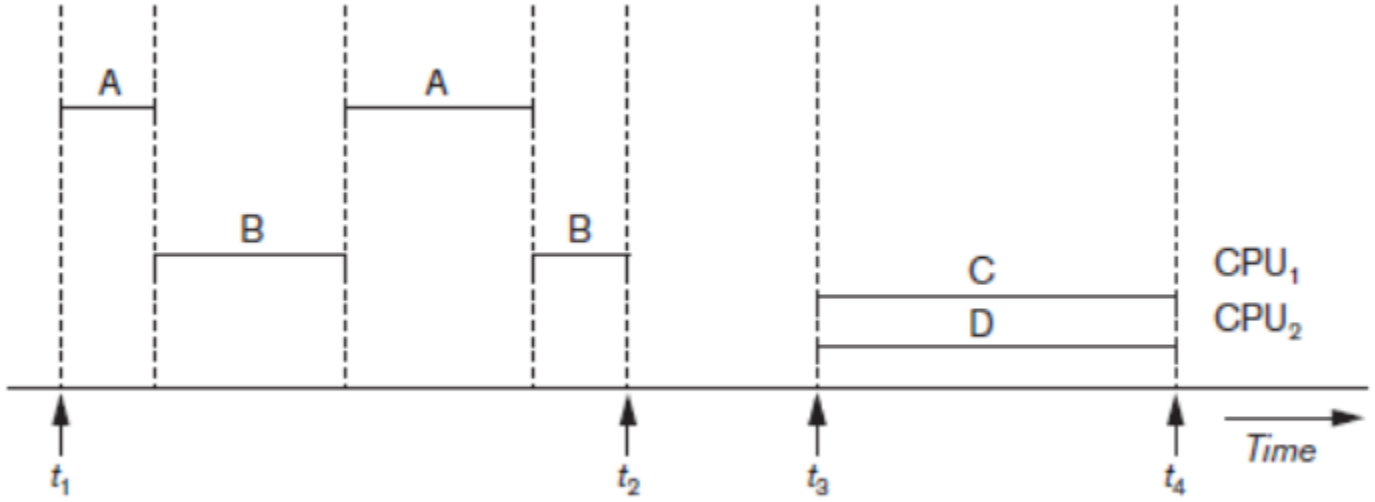



Figure 20.1 Interleaved processing versus parallel processing of concurrent transactions

# Introduction to Transaction Processing

## What is a transaction?

- A transaction is a collection of operations treated as a single logical operation
  - Typically carried out by a single user or an application program
  - Reads or updates the contents of a database
- A transaction is a 'logical unit of work' on a database
  - Each transaction does something in the database
  - No part of it alone achieves anything of use or interest to a user
- Transactions are the unit of recovery, consistency, and integrity of a database
- A *transaction* is the DBMS's abstract view of a user program: a sequence of reads and writes.

# Introduction to Transaction Processing

- Transaction: is an executing program that forms a logical unit of database processing.
  - A transaction includes one or more database access operations—these can include insertion, deletion, modification (update), or retrieval operations.
  - The database operations that form a transaction can either be embedded within an application program or they can be specified interactively via a high-level query language such as SQL.
  - **specifying the transaction boundaries :** using explicit begin transaction and end transaction statements in an application program.
  - An application program may contain **several transactions** separated by the Begin and End transaction boundaries.

e.g.:

```
BEGIN TRANSACTION;  -- Start the transaction


// Perform your operations here
INSERT INTO accounts (account_id, balance) VALUES (1, 1000);
UPDATE accounts SET balance = balance - 200 WHERE account_id = 1;

// If everything is successful, commit the transaction
COMMIT;


//If an error occurs, you can roll back the transaction instead

ROLLBACK;
```

**Note: Explore how to handle transactions in Java (JDBC), Python, React, (Node.js + Express).**

# Introduction to Transaction Processing

- Transaction processing systems:
  - Systems with large databases and hundreds of concurrent users
  - Require high availability and fast response time

**Read-only transaction:** If the database operations in a transaction do not update the database but only retrieve data, the transaction is called a read-only transaction.

**Read-write transaction:** If the database operations in a transaction involve both reading (retrieving) and writing (updating) the database, such transaction is called a read-write transaction.

# Why the concept of a transaction?

- Real world events require the manipulation of multiple data items
- Examples:
  - A patient's admission to a hospital
  - Transfer of money from checking to savings
  - Adding an additional column to a table
  - Purchase of an item on a website

# Database Items

- Database represented as collection of named data items
- Size of a data item called its granularity
- Data item
  - Record → record id can be the data item name.
  - Disk block → disk block address is used as data item name.
  - Attribute value of a record → Record ID + Attribute Name is used as data item name
- Transaction processing concepts independent of item granularity

The **basic database access operations** that a transaction can include are as follows:

## Read and Write Operations

- read_item(X)
  - Reads a database item named X into a program variable named X
  - Process includes finding the address of the disk block, and copying to and from a memory buffer
- write_item(X)
  - Writes the value of program variable X into the database item named X
  - Process includes finding the address of the disk block, copying to and from a memory buffer, and storing the updated disk block back to disk

# Read and Write Operations in detail:

READ AND WRITE OPERATIONS:

- Basic unit of data transfer from the disk to the computer main memory is one block. In general, a data item (what is read or written) will be the field of some record in the database, although it may be a larger unit such as a record or even a whole block.
- read_item(X) command includes the following steps:
  - Find the address of the disk block that contains item X.
  - Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
  - Copy item X from the buffer to the program variable named X.

# Read and Write Operations in detail:

## READ AND WRITE OPERATIONS (cont.):

- **write_item(X)** command includes the following steps:
  - Find the address of the disk block that contains item X. → Find Disk Block
  - Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer). → Load into Buffer
  - Copy item X from the program variable named X into its correct location in the buffer. → Update in Memory
  - Store the updated block from the buffer back to disk (either immediately or at some later point in time). → Flush to Disk

# Read and Write Operations (cont'd.)

- Read set of a transaction
  - Set of all items read
- Write set of a transaction
  - Set of all items written

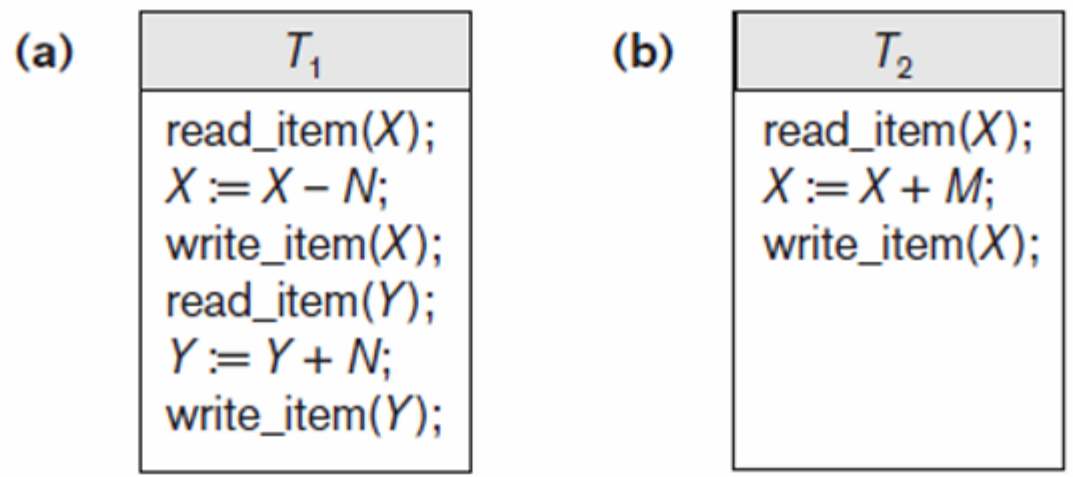



Figure 20.2 Two sample transactions (a) Transaction *T*1 (b) Transaction *T*2

# DBMS Buffers:

- In the context of a **Database Management System (DBMS)**, **buffers** are used to temporarily store data that is being transferred between **disk storage** and **main memory (RAM)**.
- Their purpose is to optimize the performance of the DBMS by reducing the number of costly disk accesses. Since **accessing data from the disk is much slower** than accessing it from memory, buffers help in improving the overall efficiency of data operations.
- A **buffer hit** occurs when the requested data is already in the buffer, and can be returned quickly. A **buffer miss** happens when the requested data is not in the buffer, requiring it to be fetched from the disk.
- Buffer pool can be managed using Replacement Policies such as **Least Recently Used (LRU), Most Recently Used (MRU).**

# DBMS Buffers

- DBMS will maintain several main memory data buffers in the database cache
- When buffers are occupied, a buffer replacement policy is used to choose which buffer will be replaced
  - Example policy: least recently used

MySQL Buffer Pool: InnoDB Buffer Pool
PostgreSQL Buffer Pool: Shared Buffers

# Why Concurrency Control is needed in a Transaction:

Transactions submitted by various users may execute concurrently. In such cases,

- Access and update the same database items
- Some form of concurrency control is needed

Furthermore, **following problems** occur when concurrent execution is uncontrolled.

# Why Concurrency Control is needed in a Transaction:

- **The Lost Update Problem**
  - This occurs when two transactions that access the same database items have their operations interleaved in a way that makes the value of some database item incorrect.
- **The Temporary Update (or Dirty Read) Problem**
  - This occurs when one transaction updates a database item and then the transaction fails for some reason (see Section 21.1.4).
  - The updated item is accessed by another transaction before it is changed back to its original value.
- **The Incorrect Summary Problem**
  - If one transaction is calculating an aggregate summary function on a number of records while other transactions are updating some of these records, the aggregate function may calculate some values before they are updated and others after they are updated.

# Why Concurrency Control is needed in a Transaction:

## Concurrent execution is uncontrolled:
## (a) The lost update problem.

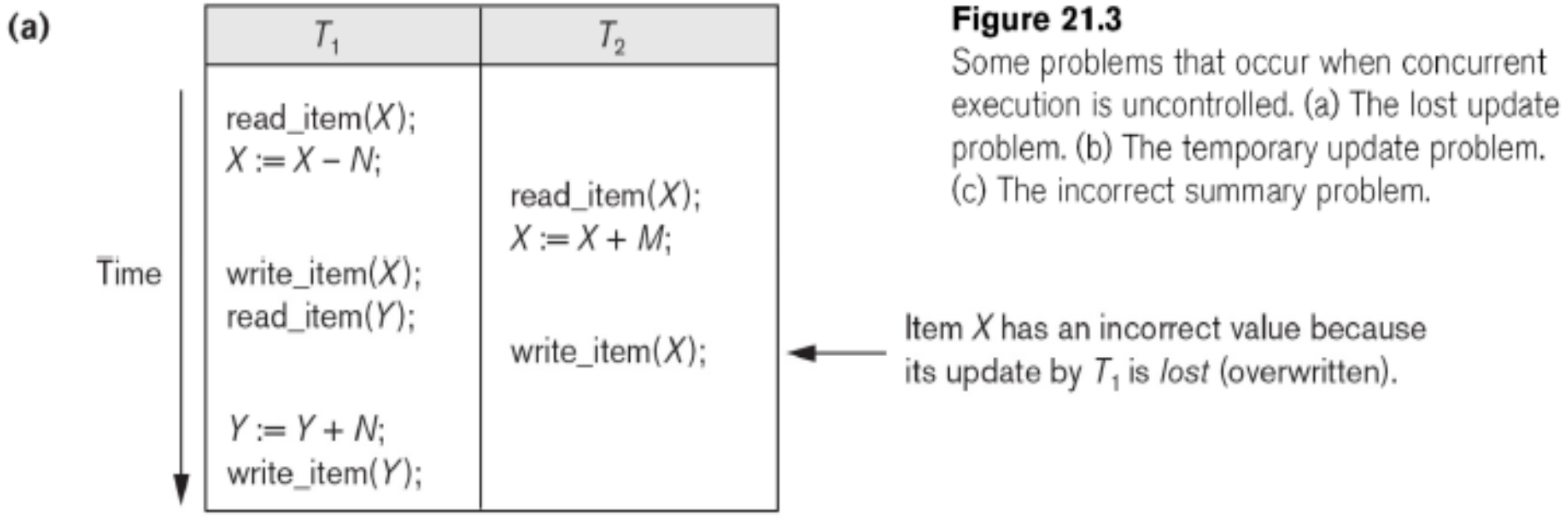



**Figure 21.3**
Some problems that occur when concurrent execution is uncontrolled. (a) The lost update problem. (b) The temporary update problem. (c) The incorrect summary problem.

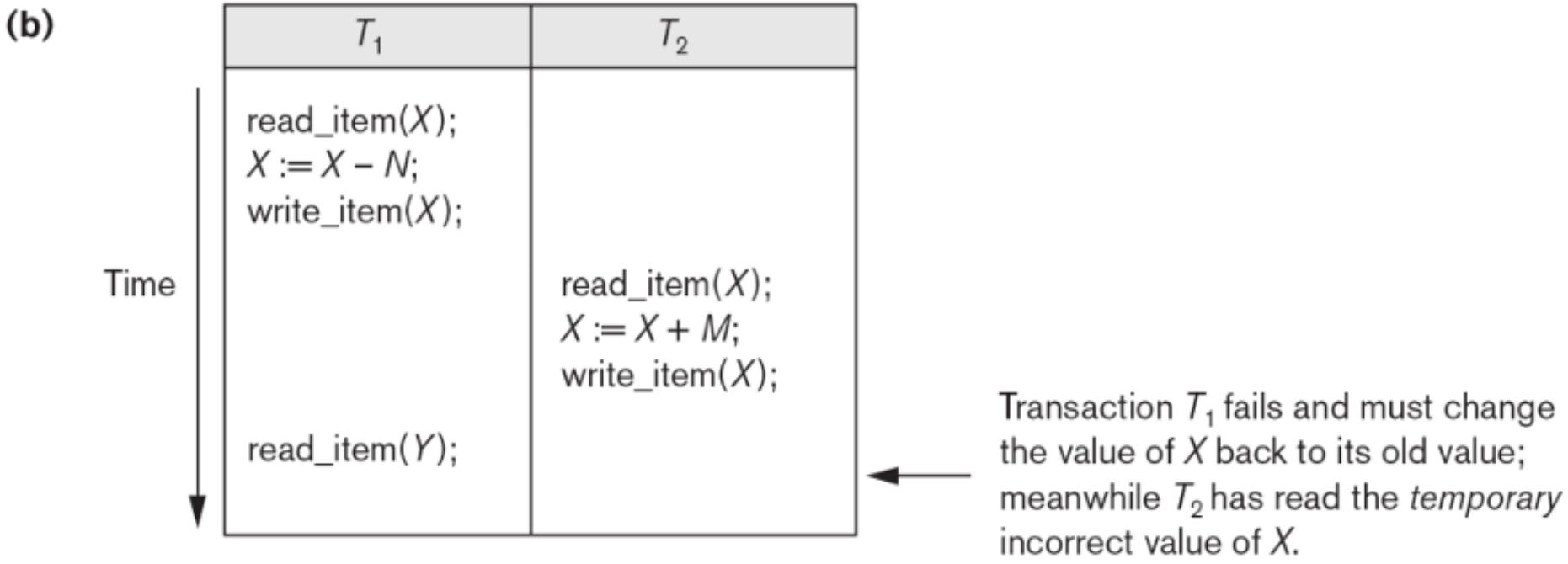
# Why Concurrency Control is needed in a Transaction:

## The lost update problem

- Occurs when two transactions that access the same database items have operations interleaved
- Results in incorrect value of some database items

# Why Concurrency Control is needed in a Transaction:

## Concurrent execution is uncontrolled:
## (b) The temporary update problem.



# Why Concurrency Control is needed in a Transaction:

## Concurrent execution is uncontrolled:
## (c) The incorrect summary problem.

(c)

| $T_1$ | $T_3$ |
|---|---|
| | *sum* := 0;<br>read_item(*A*);<br>*sum* := *sum* + *A*;<br>• • • |
| read_item(*X*);<br>*X* := *X* − *N*;<br>write_item(*X*); | |
| | read_item(*X*);<br>*sum* := *sum* + *X*;<br>read_item(*Y*);<br>*sum* := *sum* + *Y*; |
| read_item(*Y*);<br>*Y* := *Y* + *N*;<br>write_item(*Y*); | |

← $T_3$ reads *X* after *N* is subtracted and reads *Y* before *N* is added; a wrong summary is the result (off by *N*).

# The Unrepeatable Read Problem

- Transaction T reads the same item twice
- Value is changed by another transaction T′ between the two reads
- T receives different values for the two reads of the same item

# Introduction to Transaction Processing (12)

Why **recovery** is needed:

(What causes a Transaction to fail)

1. A computer failure (system crash):

   A hardware or software error occurs in the computer system during transaction execution. If the hardware crashes, the contents of the computer's internal memory may be lost.

2. A transaction or system error:

   Some operation in the transaction may cause it to fail, such as integer overflow or division by zero. Transaction failure may also occur because of erroneous parameter values or because of a logical programming error. In addition, the user may interrupt the transaction during its execution.

# Why Recovery is Needed

- Committed transaction
  - Effect recorded permanently in the database
- Aborted transaction
  - Does not affect the database
- Types of transaction failures
  - Computer failure (system crash)
  - Transaction or system error
  - Local errors or exception conditions detected by the transaction

# Introduction to Transaction Processing (13)

Why **recovery** is needed (cont.):

(What causes a Transaction to fail)

3. Local errors or exception conditions detected by the transaction:

   Certain conditions necessitate cancellation of the transaction. For example, data for the transaction may not be found. A condition, such as insufficient account balance in a banking database, may cause a transaction, such as a fund withdrawal from that account, to be canceled.

   A programmed abort in the transaction causes it to fail.

4. Concurrency control enforcement:

   The concurrency control method may decide to abort the transaction, to be restarted later, because it violates serializability or because several transactions are in a state of deadlock (see Chapter 22).

# Why Recovery is Needed (cont'd.)

- Types of transaction failures (cont'd.)
  - Concurrency control enforcement
  - Disk failure
  - Physical problems or catastrophes
- System must keep sufficient information to recover quickly from the failure
  - Disk failure or other catastrophes have long recovery times

# Introduction to Transaction Processing (14)

Why **recovery** is needed (cont.):

(What causes a Transaction to fail)

5. Disk failure:

   Some disk blocks may lose their data because of a read or write malfunction or because of a disk read/write head crash. This may happen during a read or a write operation of the transaction.

6. Physical problems and catastrophes:

   This refers to an endless list of problems that includes power or air-conditioning failure, fire, theft, sabotage, overwriting disks or tapes by mistake, and mounting of a wrong tape by the operator.

# Transaction and System Concepts

- A **transaction** is an atomic unit of work that is either completed in its entirety or not done at all.
  - For recovery purposes, the system needs to keep track of when the transaction starts, terminates, and commits or aborts.
- **Transaction states**:
  - Active state
  - Partially committed state
  - Committed state
  - Failed state
  - Terminated State

# Transaction and System Concepts

**System must keep track of when each transaction starts, terminates, commits, and/or aborts**

- BEGIN_TRANSACTION
- READ or WRITE
- END_TRANSACTION
- COMMIT_TRANSACTION
- ROLLBACK (or ABORT)

# Transaction and System Concepts

Recovery manager keeps track of the following operations:

- **begin_transaction**: This marks the beginning of transaction execution.
- **read** or **write**: These specify read or write operations on the database items that are executed as part of a transaction.
- **end_transaction**: This specifies that read and write transaction operations have ended and marks the end limit of transaction execution.
  - At this point it may be necessary to check whether the changes introduced by the transaction can be permanently applied to the database or whether the transaction has to be aborted because it violates concurrency control or for some other reason.

# Transaction and System Concepts

Recovery manager keeps track of the following operations (cont):

- **commit_transaction**: This signals a successful end of the transaction so that any changes (updates) executed by the transaction can be safely committed to the database and will not be undone.
- **rollback** (or **abort**): This signals that the transaction has ended unsuccessfully, so that any changes or effects that the transaction may have applied to the database must be undone.

# Transaction and System Concepts

Recovery techniques use the following operators:

- **undo**: Similar to rollback except that it applies to a single operation rather than to a whole transaction.
- **redo**: This specifies that certain *transaction operations* must be *redone* to ensure that all the operations of a committed transaction have been applied successfully to the database.

# State Transition Diagram Illustrating the States for Transaction Execution

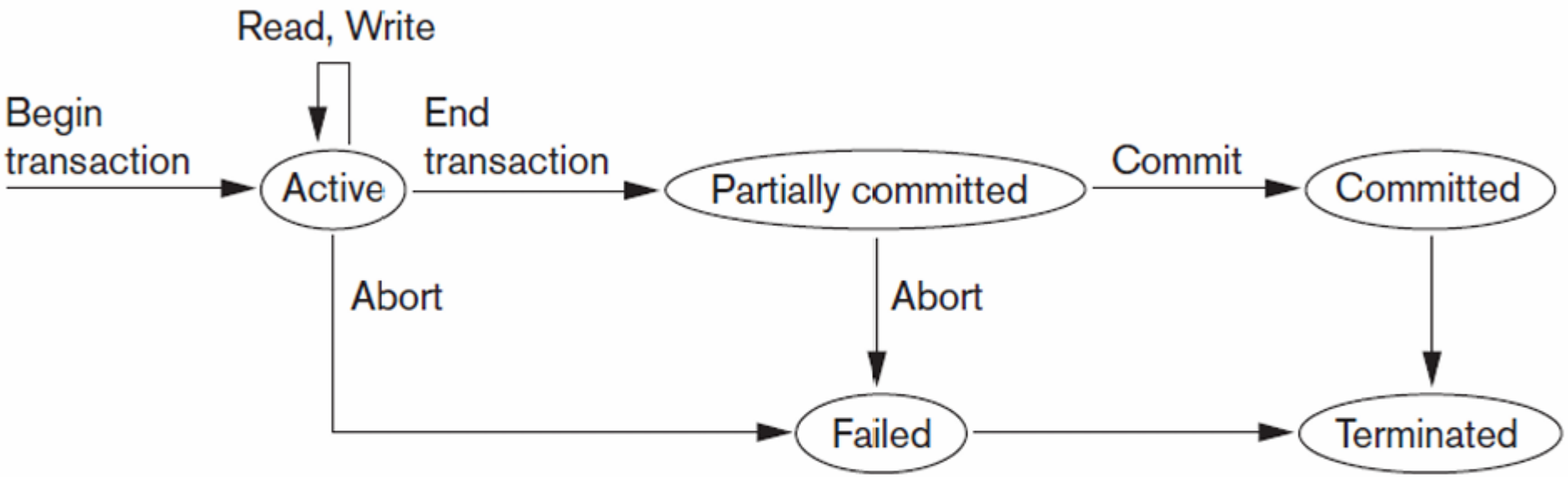



Figure 20.4 State transition diagram illustrating the states for transaction execution

# Transaction and System Concepts

- System log keeps track of transaction operations
- Sequential, append-only file
- Not affected by failure (except disk or catastrophic failure)
- Log buffer
  - Main memory buffer
  - When full, appended to end of log file on disk
- Log file is backed up periodically
- Undo and redo operations based on log possible

# Transaction and System Concepts

- The System Log
  - **Log** or **Journal**: The log keeps track of all transaction operations that affect the values of database items.
    - This information may be needed to permit recovery from transaction failures.
    - The log is kept on disk, so it is not affected by any type of failure except for disk or catastrophic failure.
    - In addition, the log is periodically backed up to archival storage (tape) to guard against such catastrophic failures.

# Transaction and System Concepts

## The System Log (cont):

- T in the following discussion refers to a unique **transaction-id** that is generated automatically by the system and is used to identify each transaction:
- Types of log record:
  - [start_transaction,T]: Records that transaction T has started execution.
  - [write_item,T,X,old_value,new_value]: Records that transaction T has changed the value of database item X from old_value to new_value.
  - [read_item,T,X]: Records that transaction T has read the value of database item X.
  - [commit,T]: Records that transaction T has completed successfully, and affirms that its effect can be committed (recorded permanently) to the database.
  - [abort,T]: Records that transaction T has been aborted.

# Transaction and System Concepts

## The System Log (cont):

- Protocols for recovery that *avoid cascading rollbacks do not require that read operations be written to the system log*, whereas other protocols require these entries for recovery.
- Strict protocols require simpler write entries that do not include new_value (see Section 21.4).

# Transaction and System Concepts

Recovery using log records:

- If the system crashes, we can recover to a consistent database state by examining the log and using one of the techniques described in Chapter 19.
  1. Because the log contains a record of every write operation that changes the value of some database item, it is possible to **undo** the effect of these write operations of a transaction T by tracing backward through the log and resetting all items changed by a write operation of T to their old_values.
  2. We can also **redo** the effect of the write operations of a transaction T by tracing forward through the log and setting all items changed by a write operation of T (that did not get done permanently) to their new_values.

# Transaction and System Concepts

## Commit Point of a Transaction

- **Definition a Commit Point:**
  - A transaction T reaches its **commit point** when all its operations that access the database have been executed successfully *and* the effect of all the transaction operations on the database has been recorded in the log.
  - Beyond the commit point, the transaction is said to be committed, and its effect is assumed to be permanently recorded in the database.
  - The transaction then writes an entry [commit,T] into the log.
- **Roll Back of transactions:**
  - Needed for transactions that have a [start_transaction,T] entry into the log but no commit entry [commit,T] into the log.

# Transaction and System Concepts

## Commit Point of a Transaction

**Redoing transactions:**

- Transactions that have written their commit entry in the log must also have recorded all their write operations in the log; otherwise they would not be committed, so their effect on the database can be redone from the log entries. (Notice that the log file must be kept on disk.
- At the time of a system crash, only the log entries that have been written back to disk are considered in the recovery process because the contents of main memory may be lost.)

**Force writing a log:**

- Before a transaction reaches its commit point, any portion of the log that has not been written to the disk yet must now be written to the disk.
- This process is called force-writing the log file before committing a transaction.

# Desirable Properties of Transactions

ACID properties:

- **Atomicity**: A transaction is an atomic unit of processing; it is either performed in its entirety or not performed at all.
- **Consistency preservation**: A correct execution of the transaction must take the database from one consistent state to another.
- **Isolation**: A transaction should not make its updates visible to other transactions until it is committed; this property, when enforced strictly, solves the temporary update problem and makes cascading rollbacks of transactions unnecessary (see Chapter 21).
- **Durability or permanency**: Once a transaction changes the database and the changes are committed, these changes must never be lost because of subsequent failure.

# Example of a transaction

**Transfer $50 from account A ($200) to account B ($50)**

```
Read(A);
A -= 50;
Write(A);
Read(B);
B += 50;
Write(B);
```

Transaction

**ACID**

- **Atomicity** - shouldn't take money from A without giving it to B
- **Consistency** - money isn't lost or gained
- **Isolation** - other queries shouldn't see A or B change until transaction is completed
- **Durability** - the money does not go back to A is transaction was marked as committed

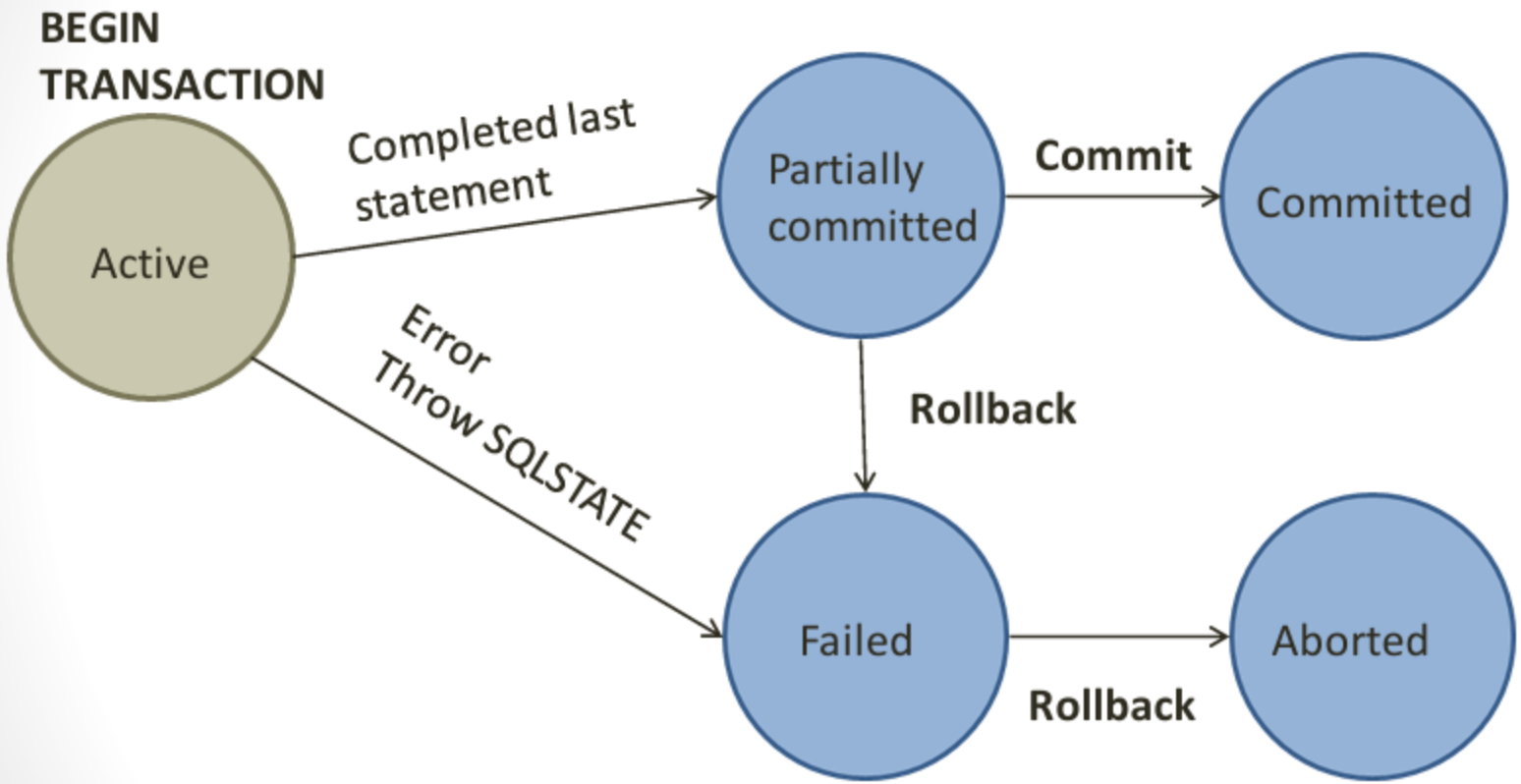
# Transaction Commands

- Begin a transaction with the My SQL command:
- **START TRANSACTION**;
- Transactions complete via 2 different input
  - **COMMIT**;
  - **ROLLBACK**;

# Transaction Outcomes

- **Can have one of two outcomes:**
  - **Success - transaction *commits* and database reaches a new consistent state.**
  - **Failure - transaction *aborts*, and database must be restored to a consistent state before it started.**
  - **Such a transaction is *rolled back* or *undone*.**
- **Committed transaction cannot be aborted.**
- **An aborted transaction that is rolled back can be restarted later.**

# Life of a transaction via a FSM




Simple state machine should help prove the correctness of algorithm

# Transaction Abort/Rollback

- Rollback signals the unsuccessful end of a transaction
- Returns the system to the state it was in before the transaction began
- System state must be the same as if the transaction had never existed
- Must abort any transactions that depend on the outcome of the aborting transaction

# Transaction Commit

- COMMIT signals the successful end of a Transaction
  - Any changes made by the transaction should be saved
  - These changes are now visible to other transactions
- Declare the transaction permanently complete
- If you commit:
  - No actions should be able to move the DBMS to a state not containing the results of the transaction
  - All operations must be forever persistent in the database

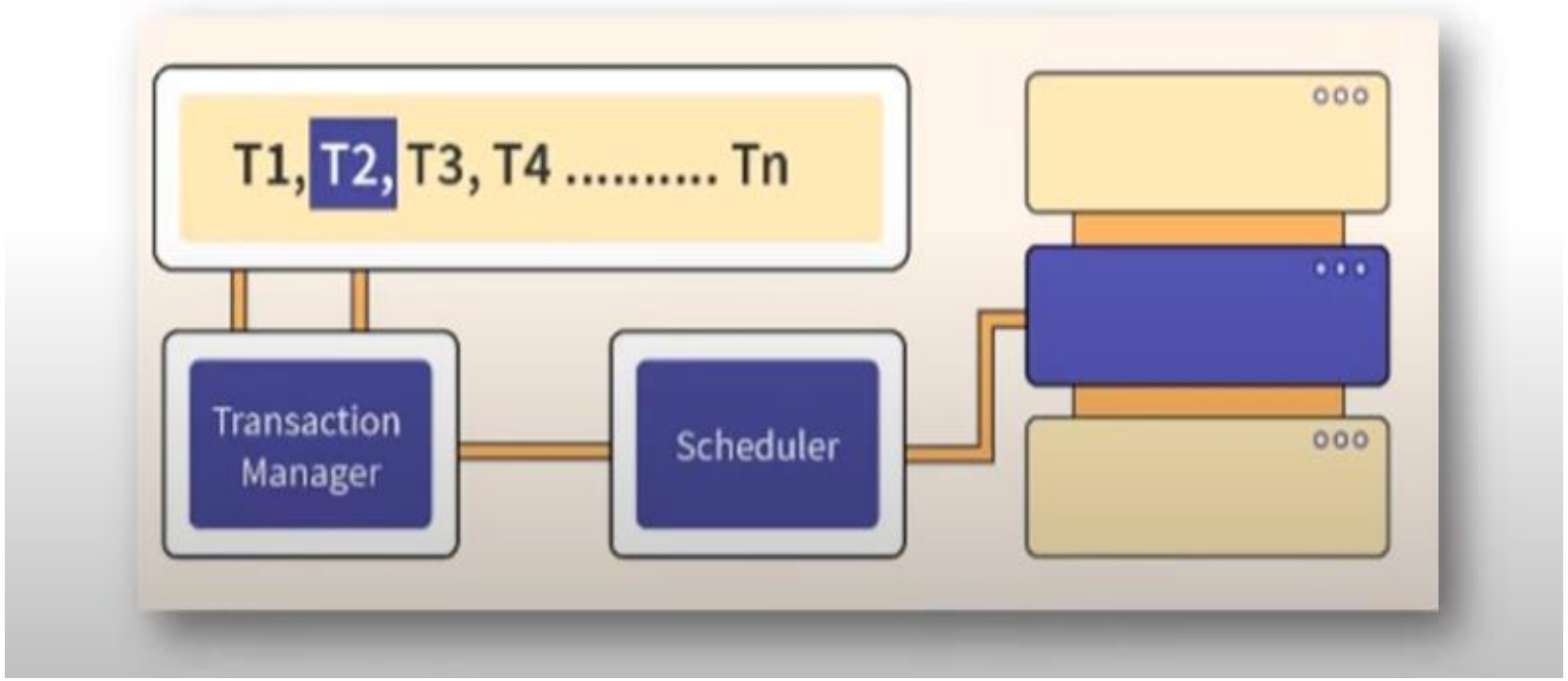
# Schedules (Histories) of Transactions

A schedule (or history) **S** of n transactions T1, T2, ... , Tn is an ordering of the operations of the transactions. Operations from different transactions can be interleaved in the schedule **S**. However, for each transaction Ti that participates in the schedule **S**, the operations of Ti in **S** must appear in the same order in which they occur in Ti.

A schedule is used to preserve the order of operations in each of the individual transaction.

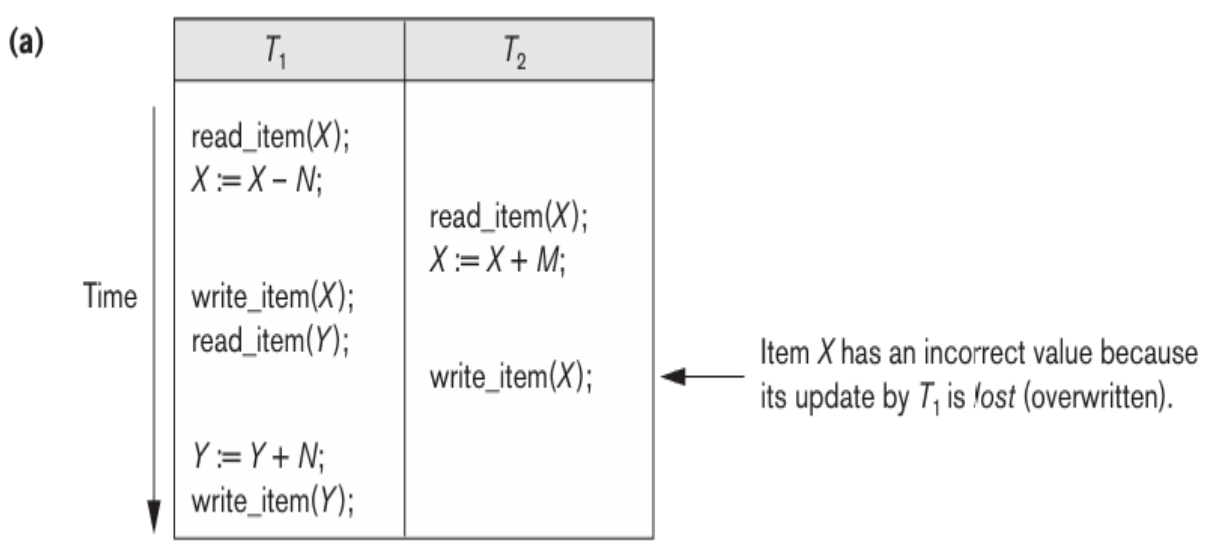
The order of operations in **S** is considered to be a **total ordering**, meaning that for any two operations in the schedule, one must occur before the other.

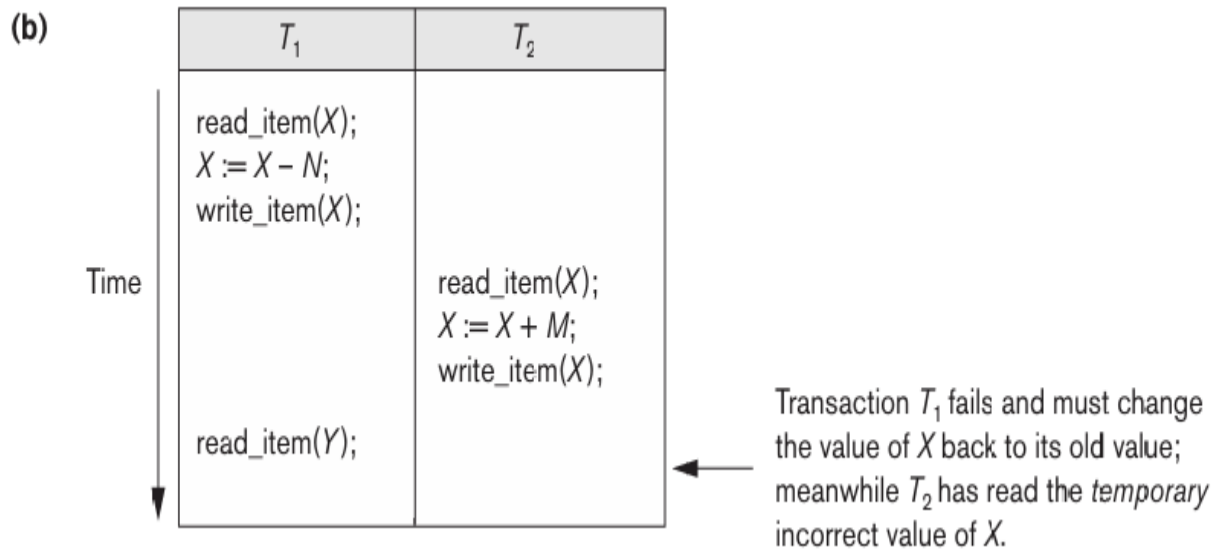
# Schedules (Histories) of Transactions




A shorthand notation for describing a schedule uses the symbols **b, r, w, e, c, and a** for the operations begin_transaction, read_item, write_item, end_transaction, commit, and abort, respectively, and appends as a subscript the **transaction id (transaction number)** to each operation in the schedule.

# Schedules (Histories) of Transactions




The schedule, **Sa** can be written as follows in this notation:

**Sa**: r1(X); r2(X); w1(X); r1(Y); w2(X); w1(Y);

Schedule, **Sb** can be written as follows, if we assume that transaction T1 aborted after its read_item(Y) operation:

**Sb**: r1(X); w1(X); r2(X); w2(X); r1(Y); a1;

# Schedules (Histories) of Transactions

## Conflicting Operations in a Schedule

Two operations in a schedule are said to conflict if they satisfy all three of the following **conditions**:

- they belong to different transactions;
- they access the same item X; and
- at least one of the operations is a write_item(X).

e.g. In **Sa**: r1(X); r2(X); w1(X); r1(Y); w2(X); w1(Y);

Conflicting operations are: r1(X) and w2(X) ;

r2(X) and w1(X), and

w1(X) and w2(X).

# Schedules (Histories) of Transactions

Non-conflicting Operations are:

r1(X) and r2(X);
w2(X) and w1(Y);
r1(X) and w1(X)

A **read-write conflict** happens when:

- One transaction **reads** a data item.
- Another transaction **writes** to the same data item.
- These operations belong to **different transactions**.

e.g.: Consider two transactions:

T1: Read(A), Write(A)

T2: Write(A), Read(A)

**Schedule :**

T1: Read(A) → (R1(A))
T2: Write(A) → (W2(A))
T1: Write(A) → (W1(A))

A **write-write conflict** occurs when two transactions attempt to write to the same data item simultaneously.

i.e., A write-write conflict happens when: Two transactions, say T1 and T2, both perform write operations on the same data item. These operations belong to different transactions.

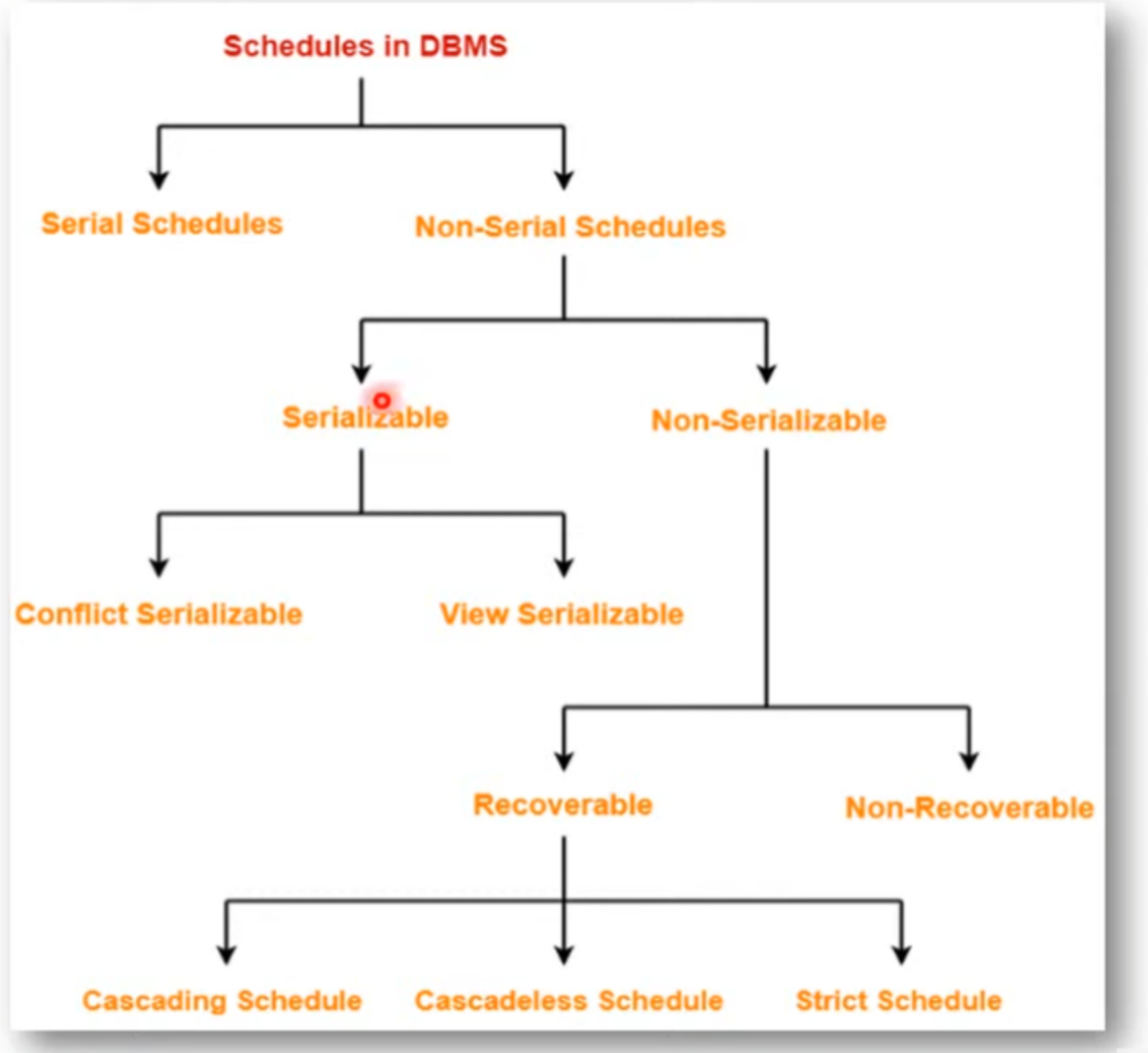
**Schedule :**

T1: Write(A = 10) → (W1(A))
T2: Write(A = 20) → (W2(A))

# Serial Schedule

- Simplest way to support transaction semantics is to require that each transaction run to completion before the next one begins
- A *schedule* is a sequence of the operations by a set of concurrent transactions that preserves the order of operations in each of the individual transactions
- A *serial schedule* is a schedule where operations of each transaction are executed consecutively without any interleaved operations from other transactions (each transaction commits before the next one is allowed to begin)
- Serial schedule - actual transaction does have the whole database to itself
- This is not a solution for the real world
  - Cannot overlap I/O operations and computation
  - Multiple cores or processors are sitting idle
  - Workload may just be really heavy
  - Response times get long as well as variable
  - Short transactions must wait for long ones to finish

# Types of Schedule



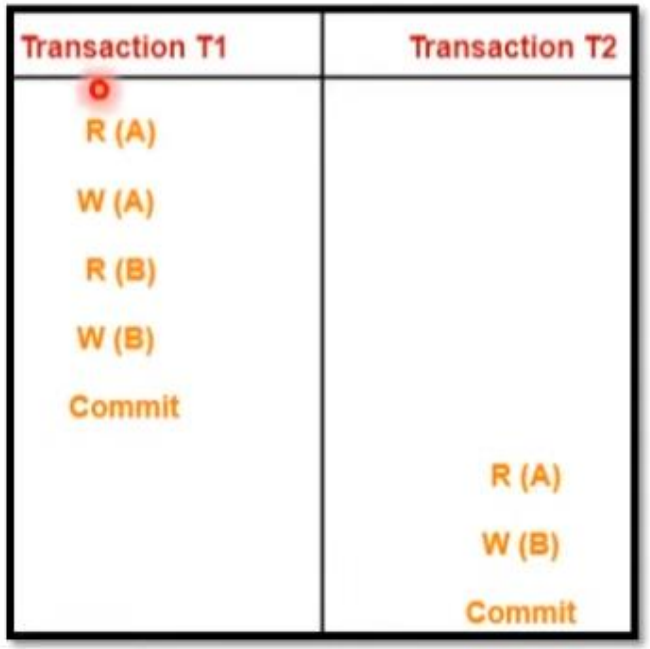
# Serial Schedule

- In serial schedule, One transection is executed completely before starting another transection.
- When one transaction executes, no other transaction is allowed to execute.
- This type of execution of the transaction is also known as non-interleaved execution.

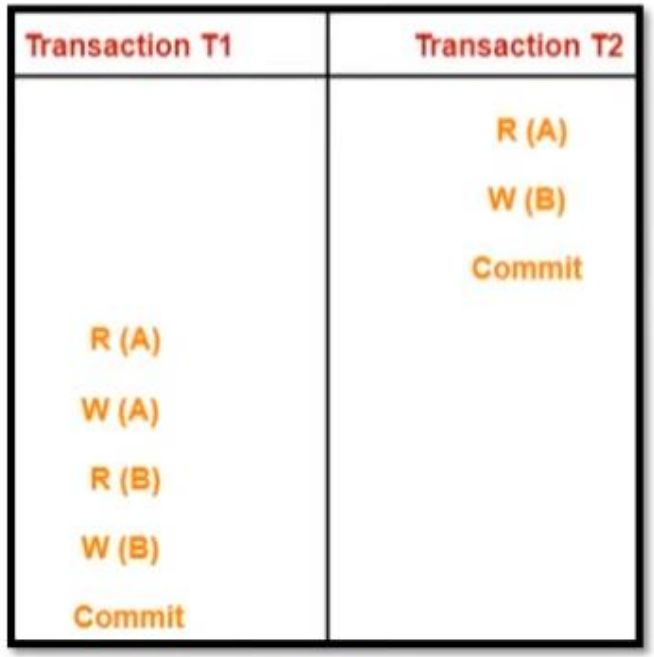
**Properties:**

**Always ->**

1. Consistent
2. Recoverable
3. Cascadeless
4. Strict




Example 1




Example 2

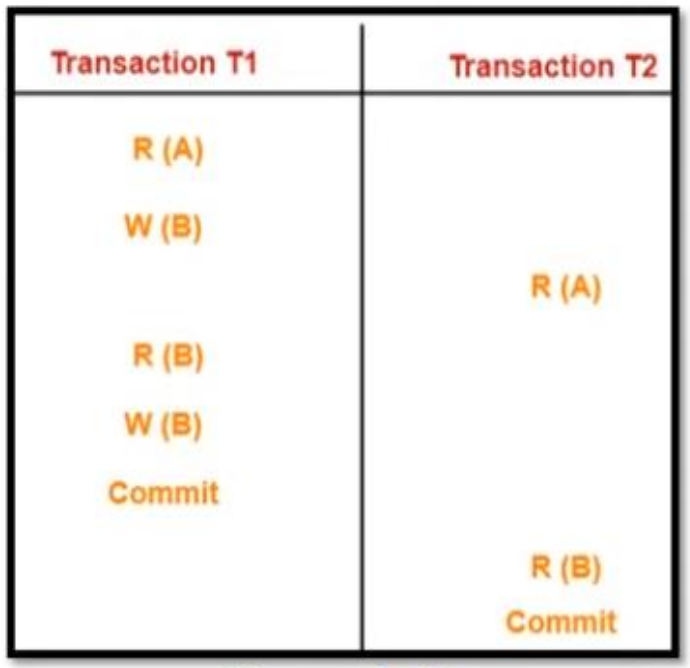
# Non - Serial Schedule

- This is a type of Scheduling where the operations of multiple transactions are interleaved.
- Operations of all the transactions are inter leaved or mixed with each other.
- This might lead to a rise in the concurrency problem.
- It has two Types: Serializable & Non Serializable Schedule.

**Properties:**

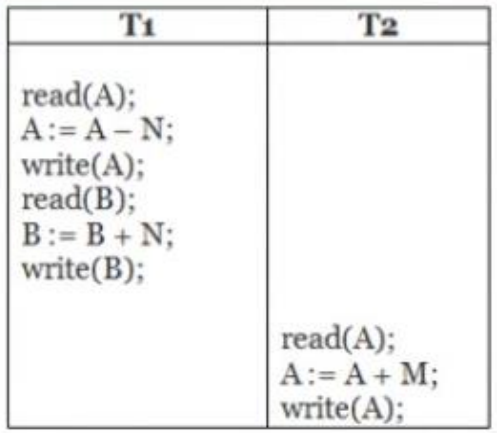
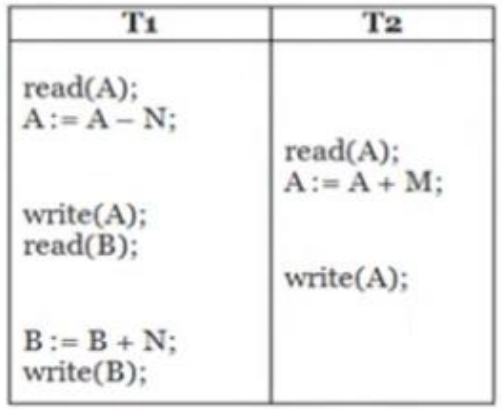
**Not Always ->**

1. Consistent
2. Recoverable
3. Cascadeless
4. Strict




**Example 1**

# Difference Between Serial & Non Serial Schedule

| No. | Serial Schedule | Non Serial Schedule |
|---|---|---|
| 1 | All transections executes one after another. | All transections execute simultaneously. |
| 2 | No Concurrency allowed. | Concurrency allowed. |
| 3 | Less resources utilization & CPU throughput. | Improve both resources utilization & CPU throughput. |
| 4 | Less efficient. | More efficient. |
| 5 | No further types. | It has two types: Serializable & Non Serializable |
| 6 | **Example:**  | **Example:**  |

# Characterizing Schedules Based on Recoverability

- **Transaction schedule or history**:
  - When transactions are executing concurrently in an interleaved fashion, the order of execution of operations from the various transactions forms what is known as a transaction schedule (or history).
- A **schedule** (or **history**) S of n transactions T1, T2, …, Tn:
  - It is an ordering of the operations of the transactions subject to the constraint that, for each transaction Ti that participates in S, the operations of T1 in S must appear in the same order in which they occur in T1.
  - Note, however, that operations from other transactions Tj can be interleaved with the operations of Ti in S.

# Characterizing Schedules Based on Recoverability

Schedules classified on recoverability:

- **Recoverable schedule**:
  - One where no transaction needs to be rolled back.
  - A schedule S is recoverable if no transaction T in S commits until all transactions T' that have written an item that T reads have committed.
- **Cascadeless schedule**:
  - One where every transaction reads only the items that are written by committed transactions.

# Characterizing Schedules Based on Recoverability

Schedules classified on recoverability (cont.):

- **Schedules requiring cascaded rollback**:
  - A schedule in which uncommitted transactions that read an item from a failed transaction must be rolled back.
- **Strict Schedules**:
  - A schedule in which a transaction can neither read or write an item X until the last transaction that wrote X has committed.

# Serial & Non Serial Schedule

- Serial schedule is always a serializable schedule because transection executed one after another.
- In Non - serial schedule transections have performed parallel.
- Only Non-serial schedule needs to be checked for Serializability

| Transaction T1 | Transaction T2 |
|---|---|
| R (A) | |
| W (A) | |
| R (B) | |
| W (B) | |
| Commit | |
| | R (A) |
| | W (B) |
| | Commit |

**Serial Schedule: S**

| Transaction T1 | Transaction T2 |
|---|---|
| R (A) | |
| W (B) | |
| | R (A) |
| R (B) | |
| W (B) | |
| Commit | |
| | R (B) |
| | Commit |

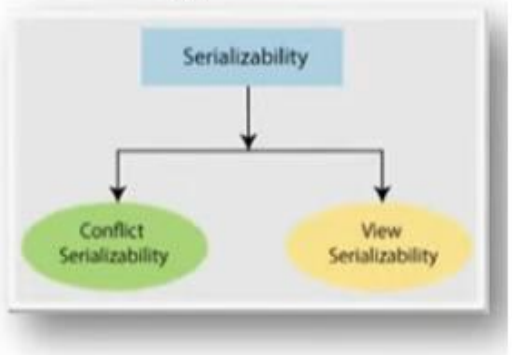
**Non- Serial Schedule: S'**

# SERIAL AND SERIALIZABLE SCHEDULE

- Serial schedule:
  - A schedule S is serial if, for every transaction T participating in the schedule, all the operations of T are executed consecutively in the schedule.
    - Otherwise, the schedule is called nonserial schedule.
- Serializable schedule:
  - A schedule S is serializable if it is equivalent to some serial schedule of the same n transactions.

# About Serializability

- Only Non-serial schedule needs to be checked for Serializability.
- Non-serial schedule is called a serializable **schedule if it can be converted to its equivalent serial schedule** OR **Result of serial & non -serial schedule is same.**
- Serializability identify correct non-serial schedules that will maintain the consistency of DB.
- Converted by using Conflict & View Serializable schedule.




| Transaction T1 | Transaction T2 |
|---|---|
| R (A) | |
| W (A) | |
| R (B) | |
| W (B) | |
| Commit | |
| | R (A) |
| | W (B) |
| | Commit |

**Serial Schedule: S**

**Converted**

**Called Serializability**

| Transaction T1 | Transaction T2 |
|---|---|
| R (A) | |
| W (B) | |
| | R (A) |
| R (B) | |
| W (B) | |
| Commit | |
| | R (B) |
| | Commit |

**Non- Serial Schedule: S'**

# Serializability

In DBMS, converting a non-serial schedule to a serial schedule is essential for ensuring consistency and correctness in concurrent transactions.

Here are the 2 **approaches**:

- **Conflict-Serializability:** If the non-serial schedule can be converted into a serial schedule by swapping non-conflicting operations.

- **View-Serializability:** If the schedule maintains the same data view as some serial schedule.

# Conflict Serializability

- A schedule is called conflict serializability if after swapping of non-conflicting operations, it can transform into a serial schedule.

**Non-serial schedule**

| T1 | T2 |
|---|---|
| Read(A) | |
| Write(A) | |
| | Read(A) |
| | Write(A) |
| Read(B) | |
| Write(B) | |
| | Read(B) |
| | Write(B) |

**Conflicting Pairs:**

1. READ(a) - WRITE(a)
2. WRITE(a) - WRITE(a)
3. WRITE(a) - READ(a)

**Conflicting Operations:**

The two operations become conflicting if all conditions satisfy:

✓Both belong to separate transactions.

✓They have the same data item.

✓They contain at least one write operation.

# Conflict Serializability

## Testing for serializability of a schedule

1. For each transaction $T_i$ participating in schedule $S$, create a node labeled $T_i$ in the precedence graph.
2. For each case in $S$ where $T_j$ executes a read_item($X$) after $T_i$ executes a write_item($X$), create an edge ($T_i \rightarrow T_j$) in the precedence graph.
3. For each case in $S$ where $T_j$ executes a write_item($X$) after $T_i$ executes a read_item($X$), create an edge ($T_i \rightarrow T_j$) in the precedence graph.
4. For each case in $S$ where $T_j$ executes a write_item($X$) after $T_i$ executes a write_item($X$), create an edge ($T_i \rightarrow T_j$) in the precedence graph.
5. The schedule $S$ is serializable if and only if the precedence graph has no cycles.

Algorithm 20.1 Testing conflict serializability of a schedule S

# Checking Whether a Schedule is Conflict Serializable Or Not

**Step 1:** Find all conflicting pairs.

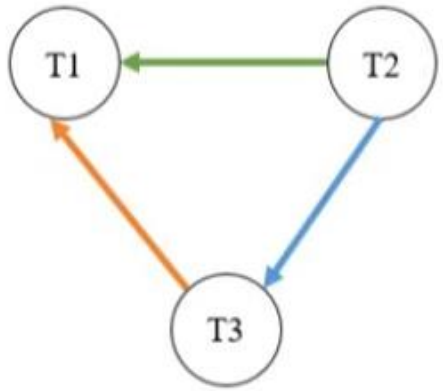
**Step 2:** Draw Precedence Graph.

| T1 | T2 | T3 |
|---|---|---|
| R(a) | | |
| | | R(b) |
| | | R(a) |
| | R(b) | |
| | R(c) | |
| | | W(b) |
| | W(c) | |
| R(c) | | |
| W(a) | | |
| W(c) | | |

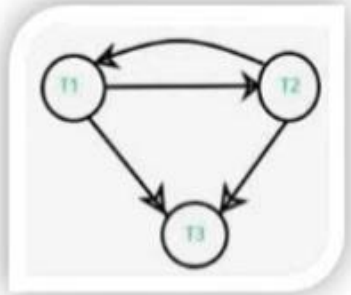
**Given Schedule S**

1. T3.R(a) – T1.W(a)
2. T2.R(b) – T3.W(b)
3. T2.R(c) – T1.W(c)
4. T2.W(c) – T1.R(c),W(C)

**Step 1: Conflict Pairs**




**Step 2: Precedence Graph**




Not Conflict Serializable Schedule

**No Loops or Cycle Present in Graph**
**Therefore,**
**Schedule is Conflict Serializable Schedule**

**Conflicting Pairs:**

1. READ(a) - WRITE(a)
2. WRITE(a) - WRITE(a)
3. WRITE(a) - READ(a)

**Rules**

# Checking Whether a Schedule is Conflict Serializable Or Not

**Step 1:** Find all conflicting pairs.

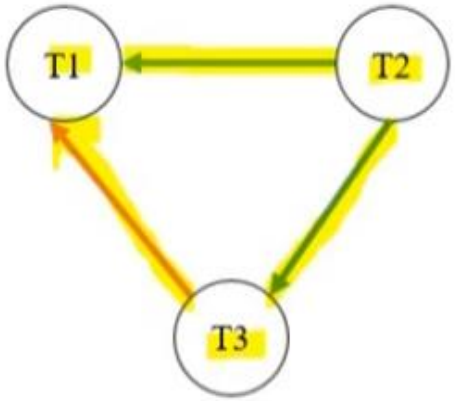
**Step 2:** Draw Precedence Graph.

| T1 | T2 | T3 |
|---|---|---|
| R(a) | | |
| | | R(b) |
| | | R(a) |
| | R(b) | |
| | R(c) | |
| | | W(b) |
| | W(c) | |
| R(c) | | |
| W(a) | | |
| W(c) | | |

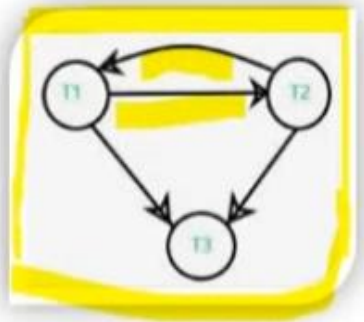
**Given Schedule S**

1. T3.R(a) – T1.W(a)
2. T2.R(b) – T3.W(b)
3. T2.R(c) – T1.W(c)
4. T2.W(c) – T1.R(c),W(C)

**Step 1: Conflict Pairs**




**Step 2: Precedence Graph**



Not Conflict Serializable Schedule

**No Loops or Cycle Present in Graph**
**Therefore,**
**Schedule is Conflict Serializable Schedule**

**Conflicting Pairs:**

1. READ(a) - WRITE(a)
2. WRITE(a) - WRITE(a)
3. WRITE(a) - READ(a)

**Rules**

# Checking Whether a Schedule is Conflict Serializable Or Not

Since there are **no cycles** in the precedence graph generated in the previous slide, the schedule is **conflict-serializable**.

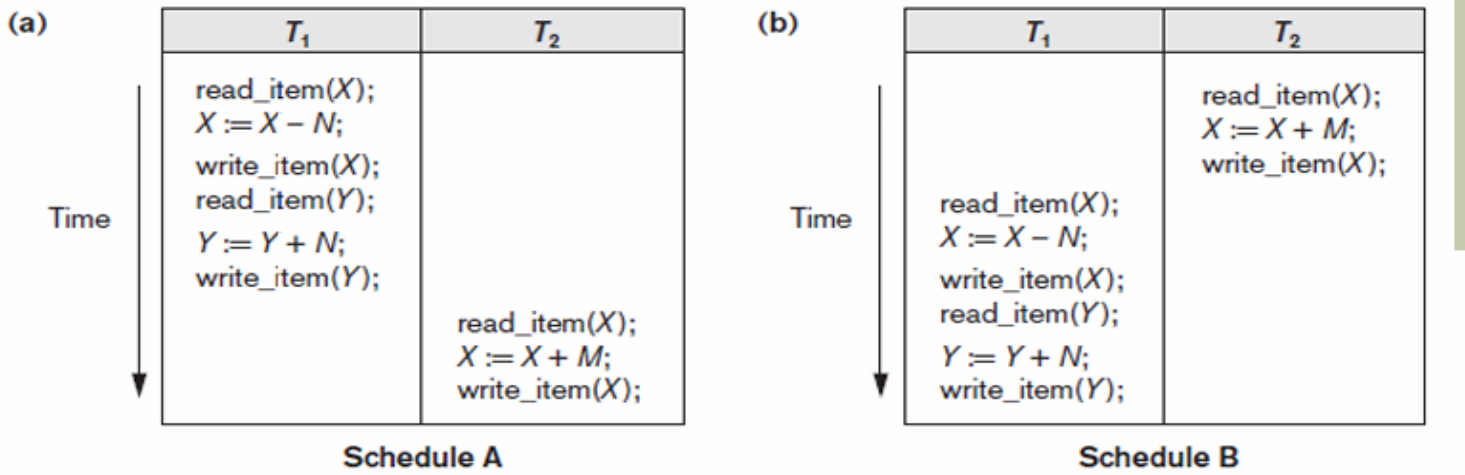
Now, we can create a serial schedule based on the **topological order of the graph**.
The precedence graph with the following edges are observed:

- **T2 → T1**
- **T2 → T3**
- **T3 → T1**

T2 must come before both T1 and T3, and T3 must come before T1.

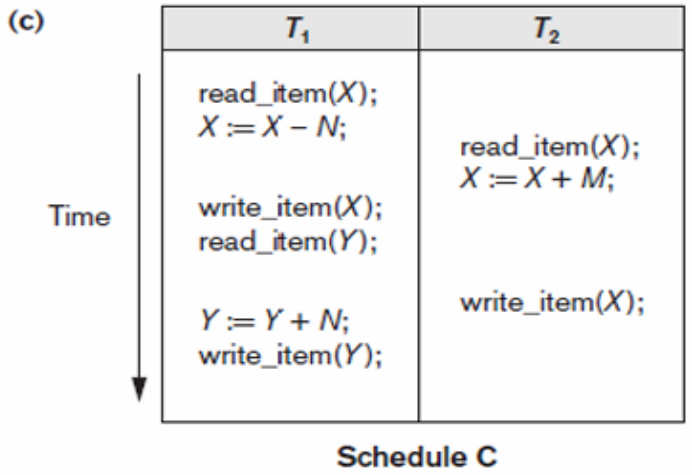
Hence, the **serial schedule is: T2 → T3 → T1** (i.e. All operations of T2, All operations of T3, All operations of T1)

# Conflict Serializability









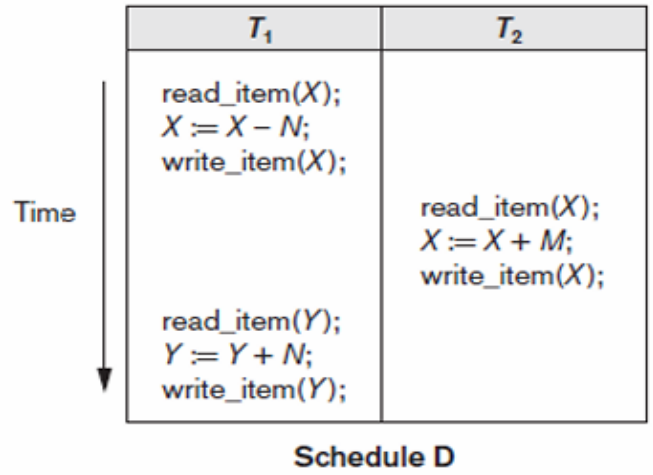




Figure 20.5 Examples of serial and nonserial schedules involving transactions $T1$ and $T2$ (a) Serial schedule A: $T1$ followed by $T2$ (b) Serial schedule B: $T2$ followed by $T1$ (c) Two nonserial schedules C and D with interleaving of operations

**Homework:**
Construct the precedence graphs for schedules **A** to **D** from Figure **20.5** to test for conflict serializability ?

# Conflict Equivalent Serializability

## Conflict Equivalent Serializability Example

**Given:**

| T1 | T2 |
|---|---|
| R(a) | |
| W(a) | |
| R(b) | |
| | R(a) |
| | W(a) |

**Schedule S**

| T1 | T2 |
|---|---|
| R(a) | |
| W(a) | |
| | R(a) |
| | W(a) |
| R(b) | |

**Schedule S'**

**Find Out:**
**If S=S'Then**
**Conflict Equivalent**

**Step 1:** Check Adjust Non Conflict Pairs as per Rules.
**Step 2:** If Present Swap it.

| T1 | T2 |
|---|---|
| R(a) | |
| W(a) | |
| | R(a) |
| | W(a) |
| R(b) | |

**Schedule S'**

Swap

| T1 | T2 |
|---|---|
| R(a) | |
| W(a) | |
| | R(a) |
| R(b) | |
| | W(a) |

**Schedule S'**

Swap

| T1 | T2 |
|---|---|
| R(a) | |
| W(a) | |
| R(b) | |
| | R(a) |
| | W(a) |

**Schedule S'**

**Hence, S = S'**
**It is Conflict Equivalent**

**Rules:**

R(A) R(A)
R(A) W(A)
W(A) R(A)
W(A) W(A)
R(B) R(A)
W(B) R(A)
R(B) W(A)
W(A) W(B)

Conflict Pairs

Non Conflict Pairs

# Conflict Equivalent Serializability

## Conflict Equivalent Serializability Example

**Given:**

| T1 | T2 |
|---|---|
| R(a) | |
| W(a) | |
| R(b) | |
| | R(a) |
| | W(a) |

**Schedule S**

| T1 | T2 |
|---|---|
| R(a) | |
| W(a) | |
| | R(a) |
| | W(a) |
| R(b) | |

**Schedule S'**

**Find Out:**
**If S=S'Then**
**Conflict Equivalent**

**Step 1:** Check Adjust Non Conflict Pairs as per Rules.
**Step 2:** If Present Swap it.

| T1 | T2 |
|---|---|
| R(a) | |
| W(a) | |
| | R(a) |
| | W(a) |
| R(b) | |

**Schedule S'**

Swap

| T1 | T2 |
|---|---|
| R(a) | |
| W(a) | |
| | R(a) |
| R(b) | |
| | W(a) |

**Schedule S'**

Swap

| T1 | T2 |
|---|---|
| R(a) | |
| W(a) | |
| R(b) | |
| | R(a) |
| | W(a) |

**Schedule S'**

**Hence, S = S'**
**It is Conflict Equivalent**

**Rules:**

- R(A) R(A)
- R(A) W(A)
- W(A) R(A)
- W(A) W(A)

→ **Conflict Pairs**

- R(B) R(A)
- W(B) R(A)
- R(B) W(A)
- W(A) W(B)

→ **Non Conflict Pairs**

# Characterizing Schedules Based on Serializability

- Serial schedule:
  - A schedule S is serial if, for every transaction T participating in the schedule, all the operations of T are executed consecutively in the schedule.
    - Otherwise, the schedule is called nonserial schedule.
- Serializable schedule:
  - A schedule S is serializable if it is equivalent to some serial schedule of the same n transactions.

# How Serializability is Used for Concurrency Control

- Being serializable is different from being serial
- Serializable schedule gives benefit of concurrent execution
  - Without giving up any correctness
- Difficult to test for serializability in practice
  - Factors such as system load, time of transaction submission, and process priority affect ordering of operations
- DBMS enforces protocols
  - Set of rules to ensure serializability

# View Equivalence and View Serializability

## View equivalence of two schedules:

- As long as each read operation of a transaction reads the result of the same write operation in both schedules, the write operations of each transaction must produce the same results.
- Read operations said to see the same view in both schedules.

- Two schedules S and S′ are said to be **view equivalent** if the following three conditions hold:

1. The same set of transactions participates in S and S′, and S and S′ include the same operations of those transactions.

2. For any operation $r_i(X)$ of Ti in S, if the value of X read by the operation has been written by an operation wj(X) of Tj (or if it is the original value of X before the schedule started), the same condition must hold for the value of X read by operation ri(X) of Ti in S′.

3. If the operation $w_k(Y)$ of Tk is the last operation to write item Y in S, then $w_k(Y)$ of Tk must also be the last operation to write item Y in S′.

# View Equivalence and View Serializability

View serializability: Definition of view serializability is based on view equivalence. i.e. A schedule is view serializable if it is ***view equivalent*** to a serial schedule.

## Are these schedules view equivalent?

| Schedule U | |
|---|---|
| T1 | T2 |
| Read (A) | |
| Write (A) | |
| | Read (A) |
| | Write (A) |
| | Read (B) |
| | Write (B) |
| Read (B) | |
| Write (B) | |

| Schedule T | |
|---|---|
| T1 | T2 |
| Read (A) | |
| Write (A) | |
| | Read (A) |
| | Write (A) |
| Read (B) | |
| Write (B) | |
| | Read (B) |
| | Write (B) |

- No – In Schedule U T2 reads initial value of B
- While in Schedule T T1 reads initial value of B

# THANK YOU